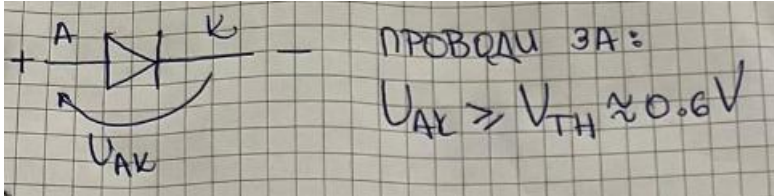


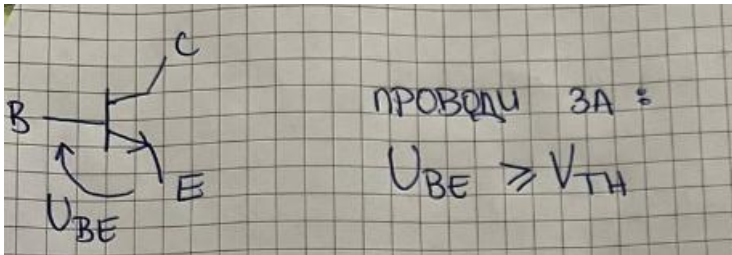
LPRS1 теорија – Скрипта за испит

1. Основни прекидачки елементи. Диода, биполарни транзистор NPN, PNP, MOSFET N канални, P канални. Критеријуми провођења, односно искључења.

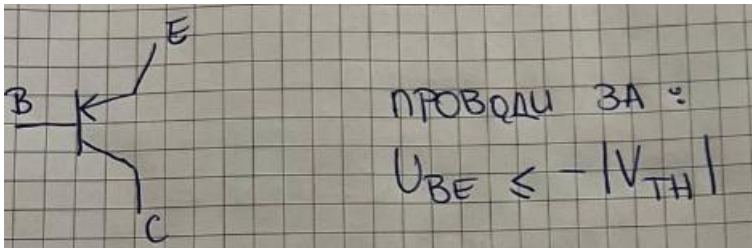
1. Диода



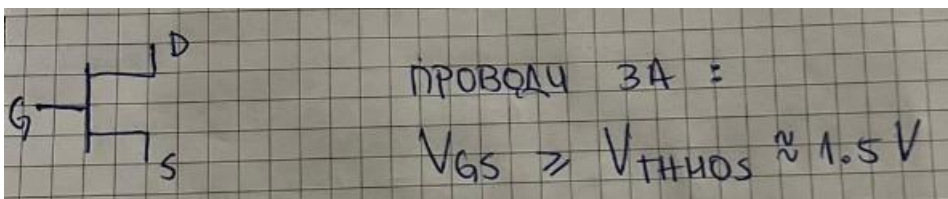
2. NPN



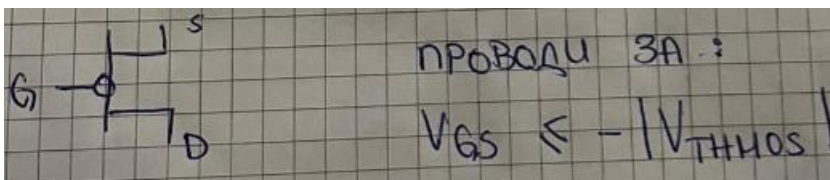
3. PNP



4. N-канални MOSFET



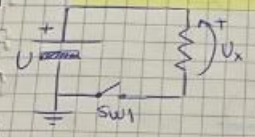
5. P-канални MOSFET



Критеријум искључења – обрнуто од укључења

2. Основна прекидачка кола.

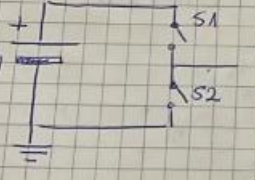
→ Прост бинарни извор:



SW1	U_x	Бин. БР	P
ОТВОРЕН	0V	"0"	0
ЗАТВОРЕН	U	"1"	U^2/R

ГЛАВА
ГДЕ СЕ
ЕНЕРГИЈА

→ ЕНЕРГЕТСКИ ЕФИКАСНОСТ РЕШЕЊЕ:



НАИЗМЕНЛИВИ ИЛИ
КОМПЛЕМЕНТАРНИ ПРЕКИДАЧИ

S1	S2	V_{OUT}	Бин. БР	ЕНЕРГИЈА
1	0	U	"1"	0
0	1	0V	"0"	0
0	0	Z		0
1	1			∞

СТАЊЕ ВИСОКЕ
ИМПЕДАНСЕ

ЗАБРАЊЕНО
СТАЊЕ

3. Реална логичка кола (сва реална лог. кола која су предавана)

НАЈЛАКШИ НАЧИН ЗА ЗАПАМТИТИ НАПАМЕТ:

I – 2 диоде улево

III – 2 диоде удесно

NE – горе P-канални, доле N-канални MOSFET

XOR – горе 2 пара P-каналних, доле 2 пара N-каналних MOSFET-а

NI – транзистор са 2 емитера; паралелно P-канални, редно N-канални MOSFET-ови

NILI – транзистори; редно P-канални, паралелно N-канални MOSFET-ови

4. Булова алгебра, (минимизација комбинационих мрежа, Карноове мапе, препознавање логичких шема и формирање функције на основу шеме, формирање логичке шеме на основу задате функције).



$$f = (\overline{AB} + C + \overline{AD}) \cdot (\overline{AB} + C + \overline{AD}) = \overline{AB} + C + \overline{AD}$$

$$= \overline{AB} + C + \overline{AD}$$

AB	00	01	11	10
00	*	*	*	*
01	*	*	*	*
11	*	*	*	*
10	*	*	*	*

$f=1 \Rightarrow \overline{AB}=1 \vee C=1 \vee \overline{AD}=1$
 $AB \in \{00, 01, 10\}$
 $AD \in \{00, 01, 10\}$
 $n_{\text{ИТАА}} < n_{\text{јединица}} \Rightarrow \text{Лакше КНФ}$

СКНФ: $f = \overline{ABCD} = \overline{A} + \overline{B} + C + D$

5. Сабирач и његова примена. Реализација сабирања и осталих арифметичких операција у VHDL-у.

Сабирач је комбинационо логичко коло које извршава операцију сабирања два или више бинарних бројева.

- **Пуни сабирач** (full-adder): 3 улаза (2 сабирка и пренос) и 2 излаза (збир и пренос).
- **Carry Lookahead сабирач** убрзава израчунавање преноса.
- **Вишеразредни сабирач** је комбинација више једнобитних пуна сабирача, који заједно сабирају вишецифрене (вишебитне) бинарне бројеве.

Примена: основни градивни блок у ALU, бројачима, мултипликаторима, процесорима итд.

У VHDL-у се сабирање може реализовати:

- директним коришћењем оператор
- описом логичких кола
- коришћењем уграђених библиотека за рад са типовима unsigned и signed.

6. Грејов код.

Посебан бинарни код у коме се суседне вредности разликују у само једном биту.

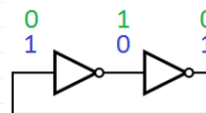
Пример (3-битни):

Децимално	Бинарно	Грејов код
0	000	000
1	001	001
2	010	011
3	011	010
4	100	110
5	101	111
6	110	101
7	111	100

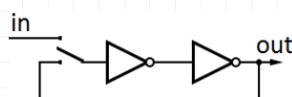
7. SR Latch, обе реализације (NI, NILI), еволуција ка DFF.

Два инвертора повезана у повратну спрегу + прекидач – најједноставнији бистабилни елемент који може памтити један бит.

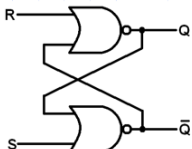
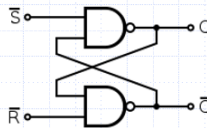
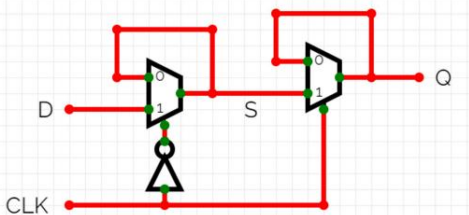
❖ Концепцијски два инвертора повезана у круг памте стање



❖ Ако би им се додао идеалан прекидач, стање би се могло менјати



❖ Прекидач
❖ горе: упиши
❖ доле: памти

SR Latch – додавањем логичких врата (NOR или NAND) добија се контролисано памћење са улазима S (Set) и R (Reset). Напомена: постоје и D и гејтовани D лечеви.	NILI <table data-bbox="756 479 932 613"><tr><th>S</th><th>R</th><th>Q</th><th>Q̄</th></tr><tr><td>0</td><td>0</td><td>Q-</td><td>Q-</td></tr><tr><td>0</td><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td><td>0</td></tr></table> NI <table data-bbox="1187 468 1362 602"><tr><th>S</th><th>R</th><th>Q</th><th>Q̄</th></tr><tr><td>0</td><td>0</td><td>Q-</td><td>Q-</td></tr><tr><td>0</td><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td><td>1</td></tr></table>	S	R	Q	Q̄	0	0	Q-	Q-	0	1	0	1	1	0	1	0	1	1	0	0	S	R	Q	Q̄	0	0	Q-	Q-	0	1	0	1	1	0	1	0	1	1	1	1
S	R	Q	Q̄																																						
0	0	Q-	Q-																																						
0	1	0	1																																						
1	0	1	0																																						
1	1	0	0																																						
S	R	Q	Q̄																																						
0	0	Q-	Q-																																						
0	1	0	1																																						
1	0	1	0																																						
1	1	1	1																																						
D Flip-Flop (DFF) – уводи се прекидно пребацивање на ивици такта, обично комбинацијом два D лача (мастер-слејв структура), чиме се добија синхрони елемент.																																									

8. Тактовани DFF.

Само при транзицији CLK (1→0) може доћи до промене A и B!

- Ако је CLK=1 (A,B)=(0,0) излазни лећ памти
- Ако је CLK=0 промена на D не афектира (A,B)

9. Синхрони и асинхрони ресет, имплементација у VHDL-у, битне карактеристике.

Синхрони ресет: делује само уз ивицу такта; стање се ресетује када дође такт.

Асинхрони ресет: делује одмах, без чекања такта.

Имплементација у VHDL-у:

Асинхрони	Синхрони
<pre> process(clk, reset) begin if reset = '1' then q <= '0'; elsif rising_edge(clk) then q <= d; end if; end process; </pre>	<pre> process(clk) begin if rising_edge(clk) then if reset = '1' then q <= '0'; else q <= d; end if; end if; end process; </pre>

Битно: асинхрони је бржи али склонији сметњама; синхрони даје уједначено понашање.

10. Стандардни логички тип сигнала у VHDL-у (могуће вредности, њих 9 са начинама коришћења).

- Неинцијализовано (U)
- Непознато (X)
- Нула (0)

- Јединица (1)
- Висока импеданса (Z)
- Слабо непознато (W)
- Слаба нула (L)
- Слаба јединица (H)
- Недефинисано (-)

11. Проблеми у комбинационим процесима у VHDL-у

- **Креирање лечева** – бочни ефекат у синтези када комбинациона мреже није покрила све условне доделе (нема default стање)
- **Непотпуна листа осетљивости** – доводи до неслагања синтезе и симулације
- **Комбинациона петља** – добија се када се у комбинационом процесу излаз из компоненте користи као улаз
- **Multiple driver** – дефинисање истог сигнала у више процеса

12. VHDL сниппети – подразумева се познавање VHDL језика тако да се може тумачити сегмент VHDL кода, која функција је имплементирана, који су евентуални пропусти у датом коду и слично.

13. Секвенцијалне мреже (Регистри, бројачи). Серијско-паралелна, паралелно-серијска конверзија.

Регистар са дозволом уписа	Померачки регистар	Бројач
<pre> process (iCLK, iRST) begin if (iRST = '1') then sREG <= (others => '0'); elsif (iCLK'event and iCLK = '1') then if (iEN = '1') then sREG <= iD; end if; end if; end process; </pre>	<pre> process (iCLK, iRST) begin if (iRST = '1') then sREG <= (others => '0'); elsif (iCLK'event and iCLK = '1') then if (iEN = '1') then if (iLOAD = '1') then sREG <= iD; else sREG <= sREG(6 downto 0) & '0'; end if; end if; end if; end process; </pre>	<pre> process (iCLK, iRST) begin if (iRST = '1') then sREG <= (others => '0'); elsif (iCLK'event and iCLK = '1') then if (iEN = '1') then if (iLOAD = '1') then sREG <= iD; else if (sREG = 199) then sREG <= (others => '0'); else sREG <= sREG + 1; end if; end if; end if; end if; end process; </pre>

- **Типичан бројач** – секвенцијално коло које броји у бинарном систему, најчешће са улазним или силазним редоследом. Користи D или T флип-флопове.
- **Децимални бројач** – броји од 0 до 9, а затим се ресетује на 0. Користи се за приказивање цифара у декадном облику.
- **Редни (кружни) бројач** – активна логичка „1“ кружи кроз низ флип-флопова; на пример: 1000 → 0100 → 0010 → 0001 → 1000.
- **Џонсонов бројач** – модификован кружни бројач где се инвертовани последњи излаз враћа на улаз првог флип-флопа. Добија се шира секвенца стања; на пример за 4 бита: 0000 → 1000 → 1100 → 1110 → 1111 → 0111 → 0011 → 0001 → 0000 – укупно 2n различитих стања.

Серијско-паралелна конверзија: Податак долази серијски (бит по бит, узастопно) и претвара се у паралелни облик (више бита истовремено, у регистру).

Паралелно-серијска конверзија: Податак је у паралелном облику (више бита одједном), а претвара се у серијски ток (бит по бит).

14. Аутомати (Милијев, Муров)

Аутомат је дигитални систем код којег излазне променљиве зависе од тренутних улазних променљивих, али и од прошлих вредности (парова улаза-излаза).

Све улазне функције које имају исти ефекат на излаз групишу се у једну **класу (стање аутомата)**.

- **Милијев аутомат** – излаз зависи од улаза и излаза и тренутног стања аутомата
- **Муров аутомат** – излаз зависи само од тренутног стања аутомата

У теорији аутомати су описани следећим величинама:

- Скуп вредности излаза
- Скуп вредности улаза
- Скуп вредности стања
- Почетно стање
- Функција прелаза (у VHDL-у је за то најпогоднија CASE архитектура)
- Функција стања (VHDL за опис овога омогућује тип енумерације)

```
architecture Behavioral of automat is

    type state_type is (S0, S1, S2);
    signal sSTATE, sNEXT_STATE : state_type;

begin

    -- Регистар за памћење стања
    process(clk, rst)
    begin
        if rst = '1' then
            sSTATE <= S0;
        elsif rising_edge(clk) then
            sSTATE <= sNEXT_STATE;
        end if;
    end process;

    -- Комбинациони процес за следеће стање
    process(sSTATE, ulaz)
    begin
        case sSTATE is
            -- <логика за sNEXT_STATE>
            when others =>
                sNEXT_STATE <= S0;
            end case;
        end process;

    end Behavioral;
```

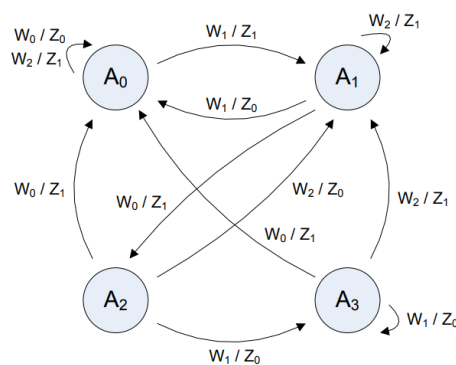
- Овде је јасно да је (0, 3) и (0, 7) класа еквиваленције (по принципу транзитивности ако „0-3 и 0-7, па следи и „3-7“). Остала поља су класе еквиваленције свако за себе.
- Дакле, имамо следеће класе:
 - Стање A0 – коме одговарају стања S0, S3 и S7
 - Стање A1 – коме одговарају стања S1 и S6

- Стање A2 – коме одговарају стања S2 i S5
- Стање A3 – коме одговара стање S4 (које није ни у једном пару) – **НЕ ЗАБОРАВИ!**
- Како је неопходно да сва стања полазног аутомата буду обухваћена неком од класа еквивалентних стања, она стања која се након овог поступка не појављују, еквивалентна су једино сама себи, односно свако од њих ће формирати посебну класу еквиваленције, као што је случај са стањем S4.
- Цртањем табеле, поступак минимизације је завршен.

	A ₀	A ₁	A ₂	A ₃
W ₀	A ₀ /Z ₀	A ₂ /Z ₁	A ₀ /Z ₁	A ₀ /Z ₁
W ₁	A ₁ /Z ₁	A ₀ /Z ₀	A ₃ /Z ₀	A ₃ /Z ₀
W ₂	A ₀ /Z ₁	A ₁ /Z ₁	A ₁ /Z ₀	A ₁ /Z ₁

b) Препознавање задатог аутомата (табеларно, матрично, графовима), аутомат задат на један од ова три начина треба умети представити на друга два.

- Табеларно задат аутомат је објашњен.
- Граф:



- Матрично (напомена: нису примери из овог задатка):

Милијев

$$C = \begin{bmatrix} - & x_0/y_0 & x_1/y_1 \\ x_0/y_2 & x_1/y_1 & - \\ - & x_1/y_0 & x_0/y_2 \end{bmatrix}$$

Муров

$$C = \begin{bmatrix} - & x_0 & x_1 \\ x_0 & x_1 & - \\ - & x_1 & x_0 \end{bmatrix}$$

Милијев аутомат: Уколико у пољу у i -тој врсти и j -тој колони стоји знак „-“, то значи да из стања S_i нема прелаза у стање S_j . Када је поље попуњено, у њему је наведен улазни сигнал X_p који изазива прелаз из стања S_i у S_j , као и излазни сигнал Y_q који се издваја при том прелазу.

Муров аутомат: Принцип је исти, али се у пољима матрице налазе само улазни сигнали за које се врши прелаз из стања S_i у S_j , јер излаз зависи искључиво од стања и није наведен у матрици.

15. Представа бројева у другом комплементу, као и неозначена представа. Представа целих и фракционих бројева.

- Неозначена представа - Само ненегативни цели бројеви (нема бита за знак)
- Други комплемент

- Омогућава позитивне и негативне целе бројеве.
- Добија се инверзијом + додавањем 1.
- MSB (први бит) = бит знака (0 позитиван, 1 негативан).

○ **Цели бројеви**

Број: 13

Бинарни запис (неозначени): 1101

- Најдеснији бит (LSB) представља $2^0 = 1$
- Следећи бит представља $2^1 = 2$
- Следећи бит представља $2^2 = 4$
- Најлевљи бит представља $2^3 = 8$

$$\text{Збир: } 8 + 4 + 0 + 1 = 13$$

○ **Фракциони бројеви**

Бинарни запис: 0.101

- Први бит после тачке представља $2^{-1} = 0.5$
- Други бит представља $2^{-2} = 0.25$
- Трећи бит представља $2^{-3} = 0.125$

За 0.101:

$$1 \times 0.5 + 0 \times 0.25 + 1 \times 0.125 = 0.625$$

16. Савремене рачунарске архитектуре.

a) Фон Нојман – compute centric:

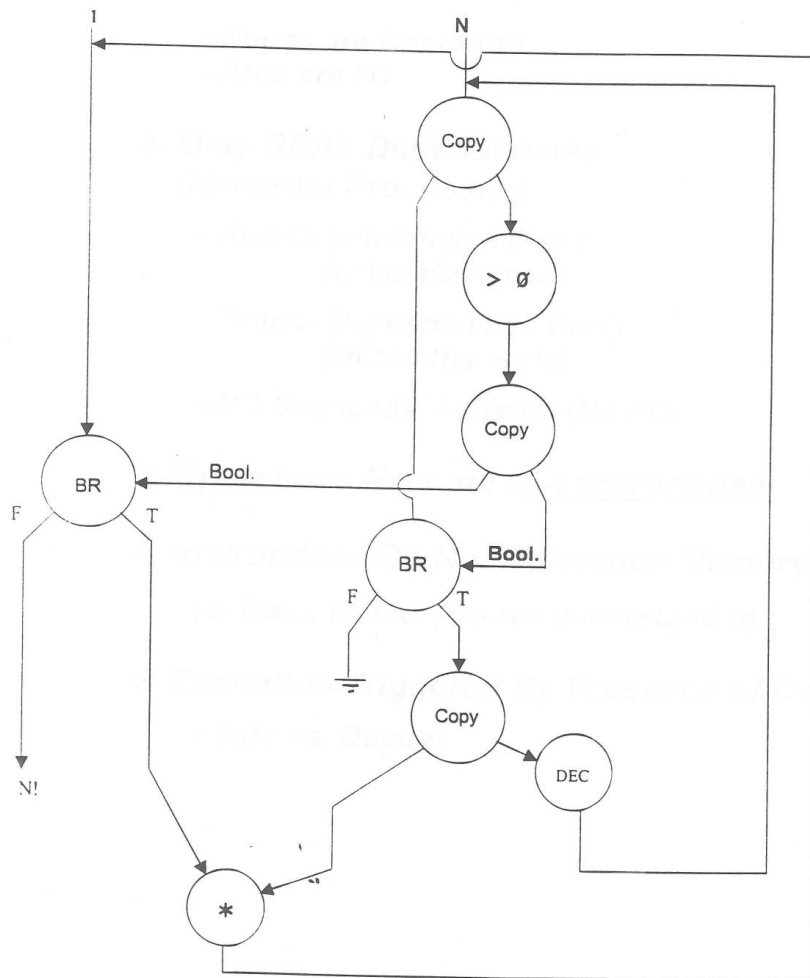
- Један меморијски простор за податке и инструкције.
- Инструкције се из меморије учитавају у процесор ради извршавања.
- Прихватање инструкција и извршавање се дешавају у оквиру редоследа контролног тока.
- Редослед извршавања је директно специфичан програмским бројачем (PC).
- Редослед је секвенцијалан, осим код инструкција које експлицитно мењају ток (нпр. jump, branch).
- Пример: класични процесори у већини PC и микроконтролера.

b) Модел тока података – data centric

- Прихват инструкције и извршавање у оквиру редоследа тока података.
- Подаци се обрађују када су спремни.
- Нема класичног програмског бројача.
- Редослед инструкција зависи од тока података.
- Свака инструкција специфицира где се прослеђује резултат.
- Инструкција се извршава чим су операнди спремни.
- Потенцијално више инструкција може се извршити паралелно.
- Инхерентан паралелизам

c) Тумачење блок дијаграма по моделу тока података

Пример - Дат је следећи блок дијаграм:



Conditional BR - инструкција условног скока (избора путање) / ако је булов резултат T (true), резултат иде лево, ако је булов резултат F (false), иде десно.

Relational > уколико је леви улаз већи, резултат је True

Тумачење: Ово је блок дијаграм по којем се рачуна факторијел од N.

1. **Улаз (N):** На улазу добијамо вредност N.
2. **Copy:** Вредност N се копира да бисмо је сачували и проследили даље.
3. **>0:** Врши се провера да ли је N веће од нуле. Овај блок производи буловску вредност (true ако је $N > 0$, иначе false).
4. **Copy:** Буловска вредност која је резултат провере се копира у следећи блок.
5. **BR (branch):** На основу буловске вредности одлучује се куда ће ток података даље. Ако је true ($N > 0$), наставља се циклус. Ако је false, иде се на излаз и враћа се тренутни резултат (N!).
6. **DEC:** Ако је услов испуњен, вредност N се декрементира за 1.
7. **Copy:** Копира се декрементирана вредност N.
8. *** (множење):** Декрементирана вредност се множи са тренутним резултатом (акумулатором). Добијени производ постаје нови резултат.
9. **Повратак на почетак циклуса:** Са новом вредношћу N процес се понавља: поново се проверава да ли је $N > 0$, док се не дође до случаја да N није веће од нуле.

d) Веза између ISA (instruction set architecture) и микроархитектуре.

- **Instruction Set Architecture (ISA)** дефинише скуп инструкција и модел који је видљив програмеру или компајлеру. То је „спољни“ опис процесора: које инструкције постоје, како изгледају регистри, како се адресира меморија, и који је редослед извршавања видљив програмеру.
- **Микроархитектура** представља конкретну реализацију те ISA у хардверу – то је „унутрашњи“ ниво који одређује како ће се те инструкције стварно извршавати.
- **Веза између њих** је у томе да микроархитектура сме да мења редослед и начин извршавања инструкција, али само ако резултат на крају остаје исти као што дефинише семантика ISA.

e) Елементи ISA (0,1,2,3 – адресне машине, опис)

- **0-адресне:** стек машине (*op, push A, pop A*)
- **1-адресне:** акумулаторске машине (*op ACC, ld A, st A*)
- **2-адресне:** машине са два операнда (*op S,D*; један се преписује)
- **3-адресне:** машине са три операнда (*op S1,S2,D*; све одвојено)

f) Елементи ISA (регистри – акумулатор са адресним регистрима, на супрот GPR)

- **Акумулатор са адресним регистрима:**
 - Рани процесори су имали акумулатор за аритметику и посебне адресне регистре само за адресирање.
 - Касније су адресни регистри добили и аритметичке функције.
- **Регистри опште намене (GPR):**
 - Сви регистри добијају општост
 - Типично их има око 32 у RISC архитектурама

g) Елементи ISA (load store на супрот memory/memory архитектуре)

- **Load/store архитектуре:** операционе инструкције раде искључиво са регистрима (већина RISC ISA)
- **Memory/memory архитектуре:** операционе инструкције могу да раде са меморијским локацијама (CISC ISA)

h) Елементи ISA (адресни модови)

- **Апсолутни** - Адреса је директно у инструкцији.
Пример: `LOAD R1, 1000` – прочитај из меморије на адреси 1000.
- **Регистарски индиректни** - Адреса је у регистру.
Пример: `LOAD R1, (R2)` – прочитај са адресе која је у R2.
- **Регистарски индиректни са офсетом** - Адреса = регистар + офсет.
Пример: `LOAD R1, 4(R2)` – адреса је R2+4.
- **Регистарски индексни** - Адреса = сабереш два регистра.
Пример: `LOAD R1, (R2+R3)`
- **Меморијски индиректни** - Узми адресу из меморије, па онда из те друге адресе податак.
Пример: `LOAD R1, M[M[R2]]`

i) Микроархитектура (једноцикрусна / вишецикрусна машина)

- **Једноцикличне:**

- Свака инструкција се извршава у једном циклусу.
- Ажурирање стања се врши на крају извршења.
- Недостатак: најспорија инструкција одређује Tclk.

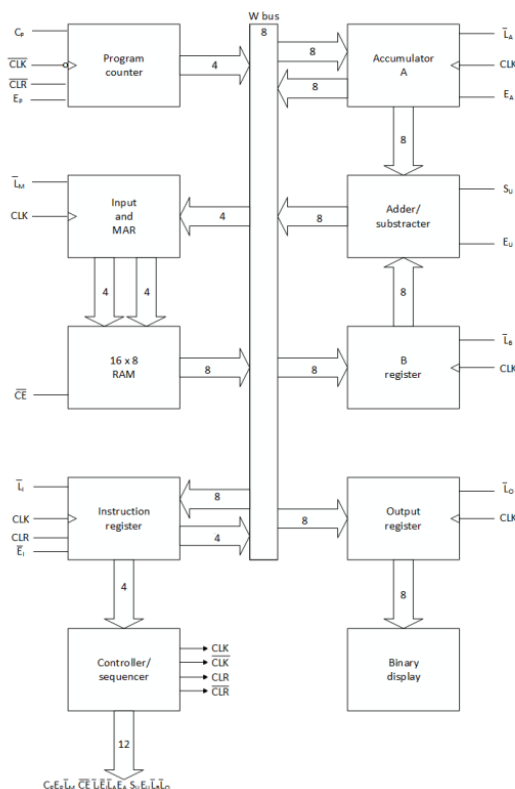
- **Вишецикличне:**

- Извршење инструкције се разлаже у више циклуса/етапа.
- Ажурирање међустања током извршења, а стања архитектуре на крају.
- Предност: Tclk је краћи јер га више не ограничава најспорија инструкција.

j) Микроархитектура (типични циклуси извршења на вишецикрусној машини)

1. Прихват (*Fetch*)
2. Декодирање (*Decode*)
3. Срачунавање адресе (*Evaluate Address*)
4. Прихват операнда (*Fetch Operands*)
5. Извршење (*Execute*)
6. Смештање резултата (*Store Result*)

18. Пример једноставног процесора “SAP” налик примеру рађеном на предавањима.



Суштина SAP процесора:

- **Једноставни процесор (Simple As-Possible Processor - SAP):** То је теоријски модел рачунара који се користи за разумевање основних принципа рада процесора.

- **Организација:** Има карактеристике он Нојманове архитектуре, где су програм и подаци смештени у истој меморији.

1. Архитектура процесора

SAP-1 процесор се састоји од кључних функционалних блокова који међусобно комуницирају:

- **Управљачка јединица:**
 - **Програмски бројач (PC):** Регистар који увек показује адресу следеће инструкције коју треба извршити.
 - **Инструкцијски регистар (IR):** Привремено чува машинску инструкцију која се тренутно извршава.
 - **Контролер – Секвенцер:** Срце управљачке јединице. Он генерише све потребне управљачке сигнале (управљачку реч), "чисте" сигнале и сигнале такта, оркестрирајући рад свих осталих компоненти процесора.
- **АЛУ (Аритметичко-логичка јединица):** Одговорна за извођење математичких и логичких операција. У SAP-1, АЛУ се састоји од:
 - **Акумулатора (AC):** Регистар опште намене који служи као главни операнд и дестинација за резултате већине аритметичких и логичких операција.
 - **Сабирача – Одузимача:** Конкретна логичка кола која изводе операције сабирања и одузимања.
 - **В регистра:** Помоћни регистар који често чува други операнд за операције у АЛУ.
- **Меморијски блок:**
 - **MAP (Memory Address Register):** Чува меморијску адресу податка или инструкције којој се тренутно приступа.
 - **16 × 8 RAM:** Радна меморија капацитета 16 локација, при чему свака локација може да чува 8 битова података или инструкција.
- **Улазно-излазна јединица (И/О):** Омогућава комуникацију процесора са спољним светом:
 - **Улазни програмабилни прекидачи (switch):** Служе за ручни унос података или команди у процесор.
 - **Излазни порт:** Преко њега процесор шаље обрађене податке или резултате.
 - **Бинарни дисплеј:** Визуелно приказује стање регистара или резултате операција у бинарном формату.

2. Пример инструкцијског сета:

- **LDA (Load Accumulator):** Учитава податак са одређене меморијске адресе у акумулатор.
- **ADD (Add):** Сабира садржај акумулатора са податком са одређене меморијске адресе.
- **STA (Store Accumulator):** Чува садржај акумулатора на одређеној меморијској адреси.
- **JMP (Jump):** Безусловни скок на одређену меморијску адресу (мења вредност PC-а).

Ispit (grupa 2)

U tabelama niže, data je sekvenca Fetch (prihvatne faze), kao i poslednje tri faze svake od instrukcija, obzirom da su početne tri mikroinstrukcije za svaku od njih ispravno Fetch (prihvat).

Sekvenca	Kontr. reč
T1	5E3 Hex
T2	0B3 Hex
T3	263 Hex

Sekvenca	Kontr. reč
T4	2B3 Hex
T5	2F1 Hex
T6	3CF Hex

Sekvenca	Kontr. reč
T4	1A3 Hex
T5	2EC Hex
T6	3E3 Hex

Sekvenca	Kontr. reč
T4	38B Hex
T5	3CA Hex
T6	3C7 Hex

Sekvenca	Kontr. reč
T4	1B5 Hex
T5	2C3 Hex
T6	5E7 Hex

Sekvenca	Kontr. reč
T4	2BR Hex
T5	2C1 Hex
T6	3CD Hex

Vodeći računa o kodovima instrukcija iz Tabele 1, popuniti adresni ROM (Address ROM) i kontrolni ROM (Control ROM) sa Slike 1, tako da realizovana UJ ispravno radi.

Поступак попуњавања контролног ROM-а и адресног ROM-а за SAP процесор:

1. Колону Адреса попуњавамо адресе редом у бинарном облику: 0000, 0001, 0010, 0011, 0100, 0101 итд.
2. У контролни ROM се на прво поље инструкцијске колоне уписује FETCH инструкција, а садржај се преписује из колоне Контролна реч FETCH табеле.
3. Остале инструкције попуњавају се произвољним редоследом (LDA, ADD, SUB, INC, итд.).
4. Инструкциона колона контролног ROM-а се преписује у инструкциону колону адресног, али без FETCH-а.
5. У колону Садржај адресног ROM-а уписује се прва адреса на којој дата инструкција почиње у контролном ROM-у.

KONTROL. ROM		ADRESNI ROM	
A.	S	A.	S
0000	5E3	0000	0001
0001	BE3	0001	0010
0010	2C3	0010	0100
0011	5E3	0011	0110
0100	2B3	0100	0111
0101	2E1	0101	1001
0110	3CF	0110	
0111	2BR		
	2C1		
	3CD		
	1A3		
	2EC		
	3E3		
	5B3		

Задатак се налази у видеу “Iprs teorija” (17:40):

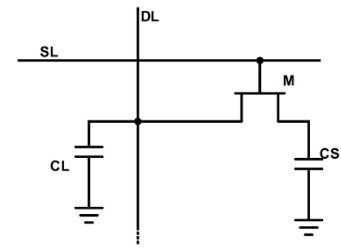
https://www.youtube.com/watch?v=aZAqTXbQ9Ys&list=PLmAj9UHbH8PPwr_LuooRiBmqAKnSpDeDV&index=1&t=2692s

19. ЛПРС1 процесор и његов асемблер.

Врло вероватно се ово односи на код, односно разумевање делова кода.

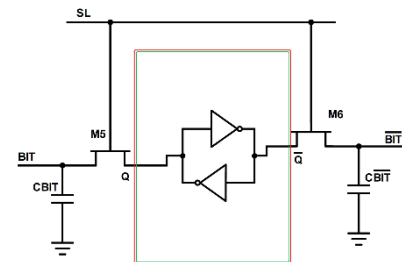
20. Логичко пројектовање меморије

а) DRAM меморијска ћелија – начин рада



ЧИТАЊЕ	УПИС
• precharge CL на $V_{dd}/2$ • Декодирање адресе, селектор укључује меморијски транзистор (M) • Паралелна веза кондензатора дели напон, sense amplifier читава да ли је $>$ или $< V_{dd}/2$ • Ако је већи, чита се '1', ако је мањи, чита се '0'	• На DL линију доведемо висок или низак напон • Декодирање адресе, селектор укључује транзистор (M) • У CS уписујемо '1' или '0' у зависности од напона на DL

б) SRAM меморијска ћелија – начин рада



ЧИТАЊЕ 1	УПИС 1
• $Q = 1, Q' = 0$ • Precharge кондензатори на V_{dd} • После декодовања адресе, напон на Cbit остаје V_{dd}, а на Cbit' опада • Sense amplifier детектује промену и чита '1'	• Почетно $Q=0$. • Поставимо $BIT=1, BIT'=0$. • После укључења селектора Q расте на 1, 'Q пада на 0 – уписана '1'.
ЧИТАЊЕ 0	УПИС 0
• $Q = 0, Q' = 1$ • Precharge кондензатори на V_{dd} • После декодовања адресе, напон на Cbit опада, а на Cbit' остаје V_{dd} • Sense amplifier детектује промену и чита '0'	• Почетно $Q=1$. • Поставимо $BIT=0, BIT'=1$. • После укључења селектора Q пада на 0, 'Q расте на 1 – уписана '0'.

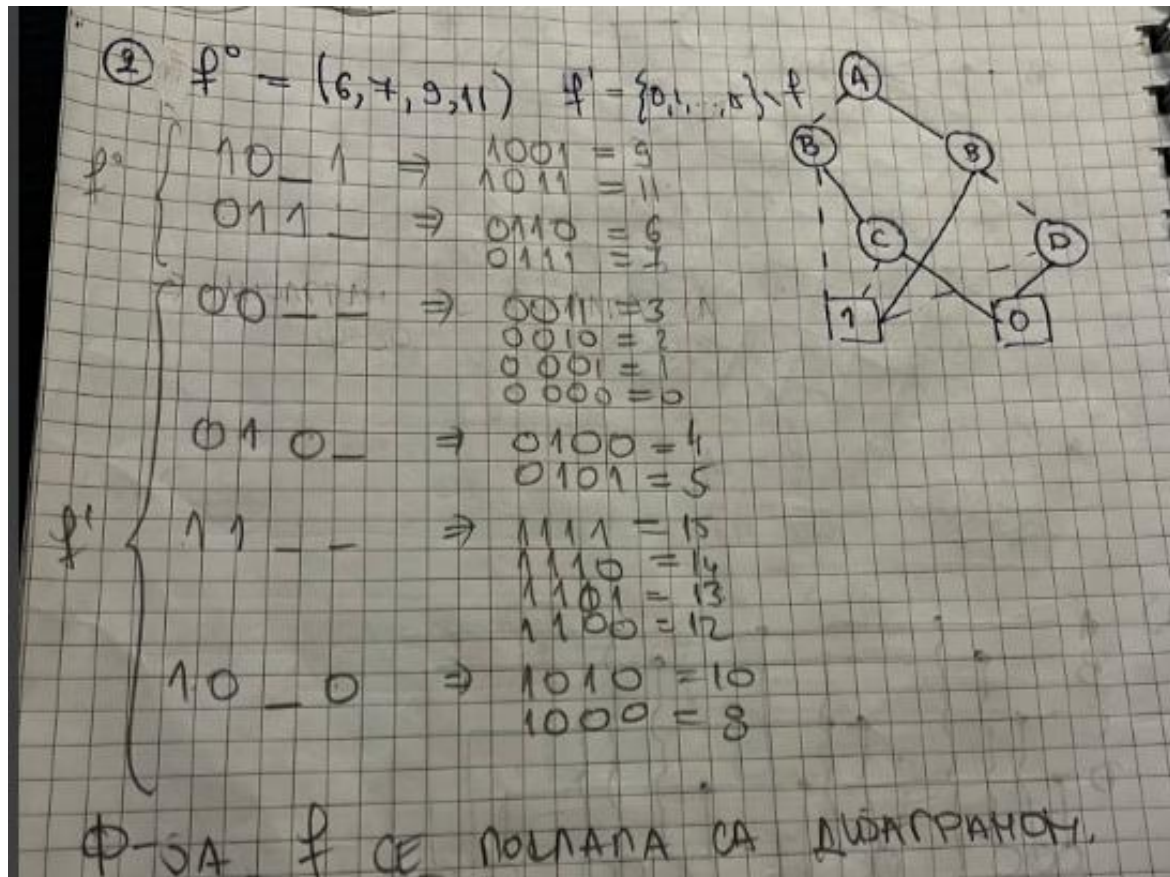
с) Магистрални приступ меморији, употреба "Z" вредности у VHDL-у.

Магистрални приступ меморији значи да више уређаја деле једну заједничку комуникациону линију — магистралу (bus) — за пренос података и адреса. Само један уређај у

одређеном тренутку може да контролише магистру (да шаље или прима податке), док остали чекају. За контролу приступа користе се сигнали дозволе као што је нпр. *enable*.

Употреба "Z" вредности у VHDL-у: "Z" означава високо-импедантно (*high impedance*) стање. Када сигнал у VHDL-у има вредност 'Z', он практично "одваја" излаз — није ни логичка 0 ни 1, већ као да није повезан. Ово је корисно за магистралне везе где више уређаја могу да деле једну линију, али само један активно управља сигналом у одређеном тренутку. Остали морају да буду у 'Z' стању да не би правили конфликт.

21. ДОДАТНО (било на првом колоквијуму): Бинарно стабло одлучивања – проверити да ли задати граф одговара задатој Буловој функцији



Објашњење: Када је $A=1$, пратимо пуну линију. Ако је $B=0$, идемо према D, а одатле се пут наставља на два начина до краја графа. Уколико је $D=0$, добијамо излаз 1. У супротном, ако је $D=1$, излаз је 0. Тако смо, на пример, дошли до бројева 1001 (што је 9) и 1011 (што је 11). На исти начин се долази и до осталих бројева. Ако се овај скуп добијених бројева подудара са вредностима функције $f(0)$, односно $f(1)$, онда се дати граф поклапа са задатом функцијом.